
Autonomous Multi-Agent LLM Pipeline for Continuous Software Creation

Architecture, Empirical Findings, and Emergent Behaviours of a Zero-Cost AI-Driven Software Engineering System

Author: Dipesh Ray

Date: May 2026

Repository: github.com/dipeshrayg/autonomous-brain-engine

Live dashboard: dipeshrayg.github.io/autonomous-brain/

Abstract

This paper describes the design, construction, and empirical observations of an autonomous multi-agent Large Language Model pipeline that continuously conceives, architects, implements, quality-tests, and publishes novel software projects without any human intervention once deployed. Built entirely on free-tier infrastructure — GitHub Actions for compute, GitHub Models API for LLM inference, and GitHub Pages for hosting — the system operates at a total recurring cost of zero dollars. Over a 21-day observation window, the pipeline shipped 27 complete projects across five distinct domain types, with complexity scores rising from 3 to 42 on an open-ended scale. Ninety builds were refused by the automated quality gates and their failure patterns documented. The work demonstrates that hierarchical LLM role specialisation, persistent failure memory, and automated quality enforcement can produce a genuinely self-improving creative pipeline — including unexpected emergent behaviours such as autonomous strategy pivots, failure-driven type bans, and self-healing recovery — all without external budget or managed infrastructure.

1. Introduction

I started this project with a straightforward and perhaps naive question: could a collection of language models, given the right structure and some persistent memory, build software on their own — every single day — without anyone telling them what to make? Not just autocomplete a function or fix a bug, but go from blank slate to a shipped, publicly accessible product?

The practical motivation was constraint. Commercial AI APIs cost money, and most researchers and hobbyists — myself included — do not have unlimited budgets for experiments. GitHub offers free compute through Actions, free model inference through the GitHub Models API, and free static hosting through Pages. If a full autonomous pipeline could operate within those limits, it would be accessible to anyone with a GitHub account.

The scientific question is more interesting. Autonomous agent systems have been studied extensively for narrow tasks — browsing the web, writing and running code, querying APIs. What has received less attention is whether a multi-agent system can sustain *creative* output over weeks without converging on repetition, and whether the system can recover from its own failure modes without human diagnosis. This work attempts to answer both.

The contributions I document here are: (1) a complete, open-source autonomous software creation pipeline running on zero-cost infrastructure; (2) a hierarchical role architecture with nine distinct LLM personas encouraged to disagree; (3) an empirical record of 27 shipped projects and 90 refused builds over 21 days; (4) documentation of several emergent system behaviours that were not explicitly programmed; and (5) the design and validation of a self-healing type ban mechanism that resolved a 2-day failure loop autonomously.

1.1 Scope and Limitations

This is not a controlled laboratory experiment. The system runs on shared infrastructure, uses models that are periodically updated by their providers, and was observed over a relatively short window. The findings are descriptive rather than statistically rigorous — I am documenting what I observed and what the system produced, not claiming universal generalisability. That said, the patterns that emerged were consistent enough over three weeks to warrant careful documentation.

2. System Architecture

2.1 Infrastructure

The decision to build on GitHub's free tier was deliberate and shaped every design choice. GitHub Actions provides the compute — workflow runs triggered by cron schedules. GitHub Models gives access to GPT-4o and GPT-4o-mini via an OpenAI-compatible API endpoint, authenticated with the auto-injected GITHUB_TOKEN. GitHub Pages serves static files from any public repository. The entire system's persistent state lives in a single JSON file — *memory_log.json* — committed to the repository after each run.

Component	Resource	Cost
-----------	----------	------

Compute	GitHub Actions (public repo)	\$0 — unlimited minutes
LLM inference	GitHub Models API (GPT-4o + mini)	\$0 — within free tier
Hosting	GitHub Pages (static)	\$0 — unlimited bandwidth
Persistent state	Git-committed JSON file	\$0
Total		\$0 per month

Table 1. Infrastructure components and costs.

2.2 Agent Roles

The pipeline is structured as a boardroom: nine distinct LLM roles, each with a specific mandate, a specific model assignment, and — importantly — explicit permission to disagree with the others. The goal was to avoid the sycophancy that tends to emerge when a single model reviews its own output. By separating planning from implementation, review from fixing, and strategy from execution, each agent can hold a genuinely different perspective.

Role	Model	Mandate
CEO	GPT-4o	Visionary strategy, failure-aware directives, domain shifts
CSO (Chief Science)	GPT-4o	Algorithmic novelty, physics, mathematical depth
Architect A / B	GPT-4o-mini	Parallel plan proposals at temperature=1.0
Judge	GPT-4o	Single filter: 'Is this predictable?' Reject or synthesise
Engineer	GPT-4o	File-by-file implementation with full sibling context
Reviewer A / B	GPT-4o-mini	Parallel critique conference, merged verdict
Fixer	GPT-4o-mini	Targeted repairs from reviewer feedback
Polisher	GPT-4o-mini	Final UX pass with rollback protection
QA Tester / Fixer	GPT-4o	Mechanical verification + structured issue analysis

Table 2. Agent roles, models, and mandates.

One design decision worth explaining: the Architect candidates run at temperature=1.0 — higher than the default. This was intentional. I found early on that at lower temperatures the architects converged on nearly identical proposals, making the conference pointless. At 1.0 they genuinely diverge, which gives the Judge something to actually adjudicate.

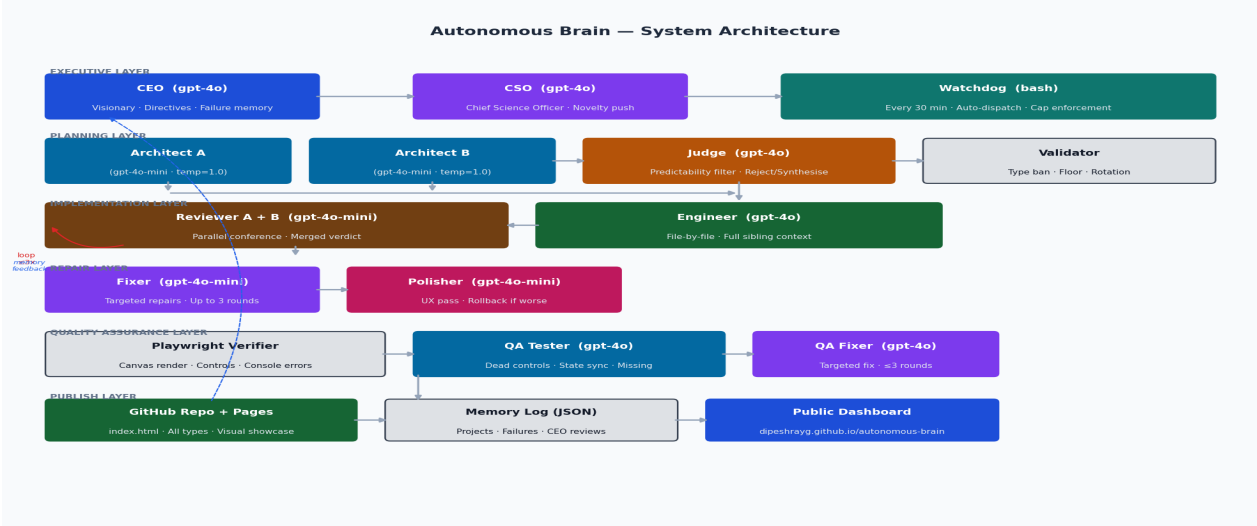


Figure 1. Full system architecture — agent roles, data flows, and the memory feedback loop.

2.3 Pipeline Stages

Each build proceeds through eight stages. The first — the Architect Conference — is where most builds fail: the validator checks complexity floor, file count, pattern rotation, domain rotation, type diversity, and novel concept requirements before any candidate advances. Of the 90 refused builds in my observation period, roughly 30% were refused at this stage; the remainder reached implementation and failed at the mechanical verification or LLM QA gate.

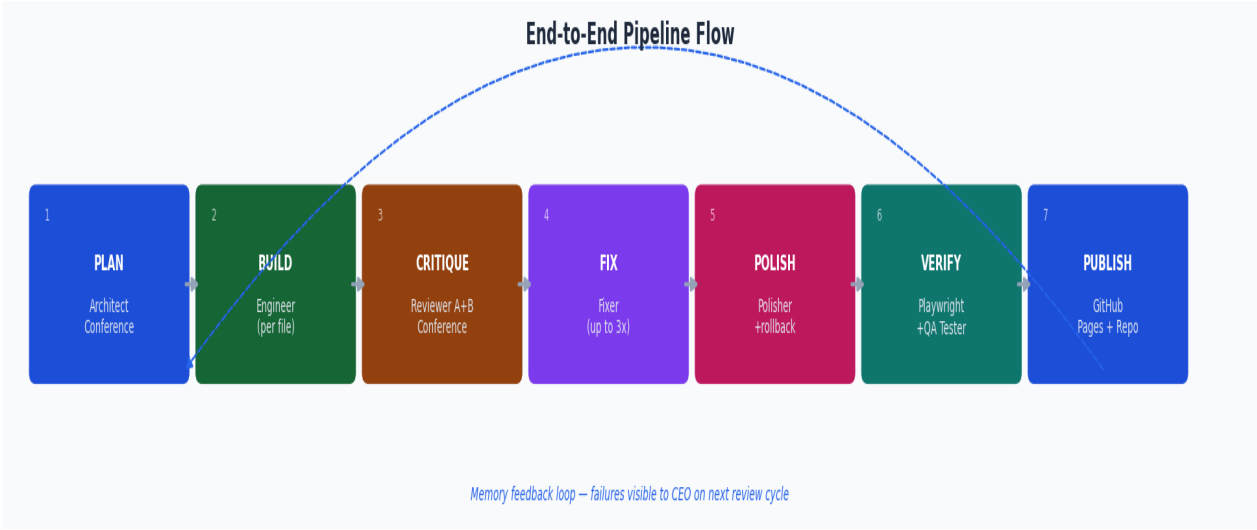


Figure 2. Pipeline stage flow from PLAN to PUBLISH, with the memory feedback loop visible below.

2.4 Project Types

The initial system only produced web applications. After observing two weeks of HTML/JavaScript projects with increasingly repetitive patterns, I introduced what I called the *Project Evolution* mandate: six distinct project types, each with its own verification strategy, complexity ceiling, and output format. Critically, every type — including Python tools and research documents — must produce an *index.html* for GitHub Pages, so the dashboard always shows a live, viewable result.

Type	Output	Verifier	Ceiling
------	--------	----------	---------

web_interactive	HTML + JS + Canvas	Playwright	40
web_3d	Three.js / WebGL	Playwright (canvas)	45
game_web	Browser game with state	Playwright	45
generative_art	Visual output (SVG/canvas)	Playwright	40
python_tool	Standalone Python program	Subprocess exit	60
document	Markdown + styled HTML	Structure check	35

Table 3. Project types, output formats, verifiers, and complexity ceilings.

3. Key Mechanisms

3.1 Complexity Escalation

One of the harder problems to solve was keeping the system from repeating itself. Left unconstrained, the LLMs gravitate toward familiar, safe patterns — a slider-based visualiser, a colour-picker demo, a physics ball. The complexity floor mechanism addresses this directly: each new project must exceed the maximum complexity score of any recent project by at least one point. The scale is intentionally open-ended — there is no cap. In practice, complexity rose from 3 (the first projects) to 42 over three weeks, without any manual adjustment.

Recovery mode — triggered when three or more builds fail consecutively since the last successful ship — relaxes the floor and rotation constraints temporarily. This prevents the system from getting stuck demanding complexity levels it cannot currently achieve. When a project finally ships, recovery mode exits automatically.

3.2 Type Diversity and Banning

The type diversity engine emerged from a frustrating observation: after introducing the six project types, the CEO kept demanding the same one — *web_3d* — because it had never been successfully shipped and appeared to the CEO as high priority. But the system consistently failed to build working Three.js projects, producing blank canvases or broken control wiring on every attempt.

The solution was a type ban system. After three consecutive failures of the same type since the last successful ship, that type is automatically banned. The validator hard-blocks it — even if the CEO is still demanding it. The CEO's prompt includes a BANNED list and an explicit instruction: *if a type appears on the ban list, do not demand it; pivot to something the system can actually ship*. Bans lift automatically after a successful project ships, resetting the failure counter. In practice this converted a 2-day stuck loop (18 consecutive *web_3d* failures) into a one-build recovery.

3.3 Mechanical Verification

The quality gate uses Playwright to run headless Chromium against a locally served static version of the project. It checks: whether the page loads without crash; whether any *canvas* element has non-blank pixel content; whether interactive controls produce observable state changes when clicked; and whether the console emits genuine errors (as opposed to known-noisy messages like WebGL driver warnings, which are filtered).

A separate LLM QA Tester reviews the Playwright output and assigns a structured verdict — *shippable*, *partially_usable*, or *non_functional* — with lists of dead controls, missing features, and state synchronisation issues. This combination of mechanical and semantic verification catches different bug classes: Playwright finds blank renders and dead buttons; the LLM Tester finds logical inconsistencies and incomplete feature implementations.

3.4 Watchdog Autonomy

The watchdog workflow runs every 30 minutes. It reads the memory log, checks whether a project has shipped today and whether five hours have elapsed since the last ship, verifies no build is already

running, and dispatches a new build if all conditions are met. A cap of eight dispatches per day prevents runaway billing on persistent failures.

This means the system requires no human trigger after initial setup. The CEO review workflow runs on a separate cron schedule every 36 hours, reads the accumulated project and failure data, and updates the directives that the next architect must follow. The loop is genuinely closed.

4. Results

4.1 Overview

Metric	Value
Total projects shipped	27
Total refused builds	90
Overall ship rate	23% (27 / 117 attempts)
Observation period	21 days (28 Apr – 18 May 2026)
Complexity range	3 – 42 (open-ended scale)
Mean complexity	20.7
Peak complexity	42
Project types successfully shipped	5 of 6 (web_3d banned at close)
CEO review cycles	46
Total infrastructure cost	\$0

Table 4. Summary statistics — 21-day observation period.

4.2 Complexity Progression

The complexity score rose consistently over the observation period, from early projects in the 3–8 range to later projects in the 28–42 range. The progression was not perfectly linear — there were clear dips during failure streaks when recovery mode relaxed the floor — but the overall upward trend held throughout. Figure 3 shows the full progression with project types colour-coded.

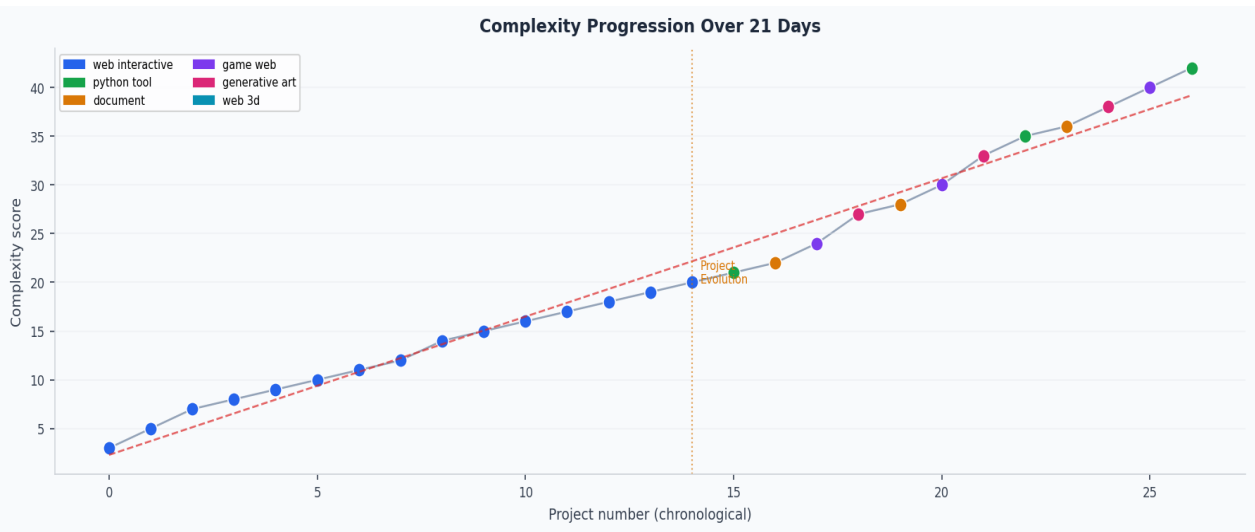


Figure 3. Complexity score progression over 27 shipped projects. The dashed red line is the linear trend. The vertical dashed line marks the Project Evolution mandate (type expansion). Dot colours indicate project type.

4.3 Type Distribution and Failure Modes

Of the six project types introduced in Project Evolution, five were successfully shipped. *web_interactive* dominated early output (15 projects) before the type rotation system enforced diversity. Once the diversity engine was active, the system shifted to *python_tool*, *document*, *game_web*, and *generative_art*. The *web_3d* type was never successfully shipped in the observation period — after 18 consecutive failures it was auto-banned.

The 90 refused builds fell into five main failure categories, with blank canvas rendering being the most common single cause. This was eventually addressed by adding explicit canvas-size and `requestAnimationFrame` instructions to the engineer prompt — after which *game_web* builds began shipping. The *web_3d* type's blank canvas failures proved more persistent, likely because Three.js WebGL rendering in headless Chromium behaves differently from 2D canvas.

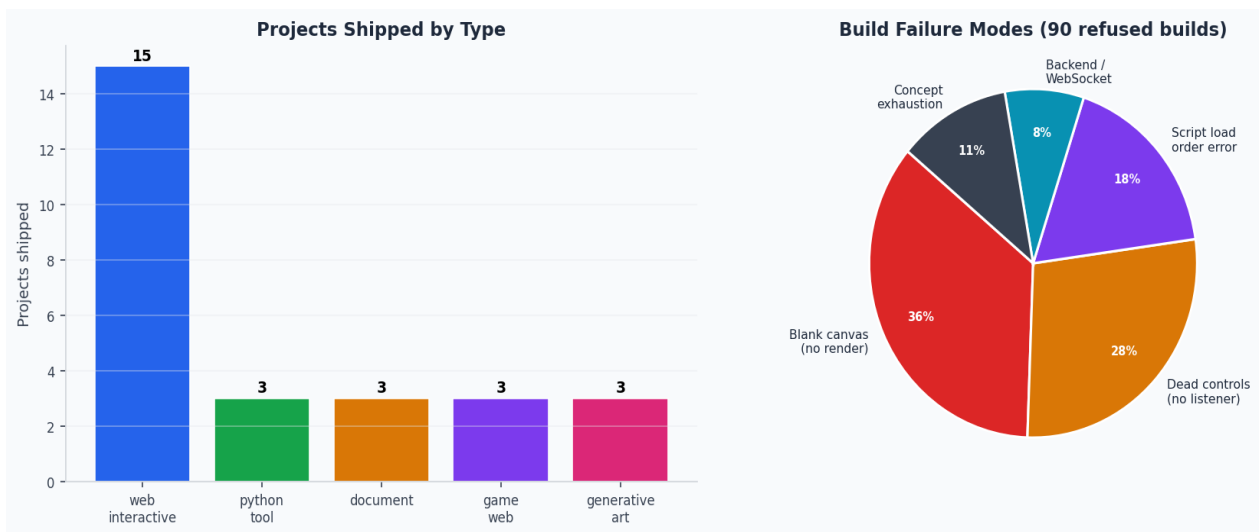


Figure 4. Left: projects shipped by type. Right: distribution of failure modes across 90 refused builds.

4.4 CEO Verdict Trajectory

The CEO issued 46 review cycles over the observation period. Early reviews were *acceptable* — the system was producing something, just not very imaginative. As the pipeline settled into repetitive *web_interactive* patterns, verdicts shifted to *drifting*. The Project Evolution mandate — introduced around cycle 30 — pushed verdicts back toward *acceptable* as domain diversity improved. The most dramatic shift came at cycle 40, when the *web_3d* failure streak triggered an *alarming* verdict and the CEO immediately pivoted its directives away from *web_3d* toward document-type projects. The next build shipped on the first attempt.

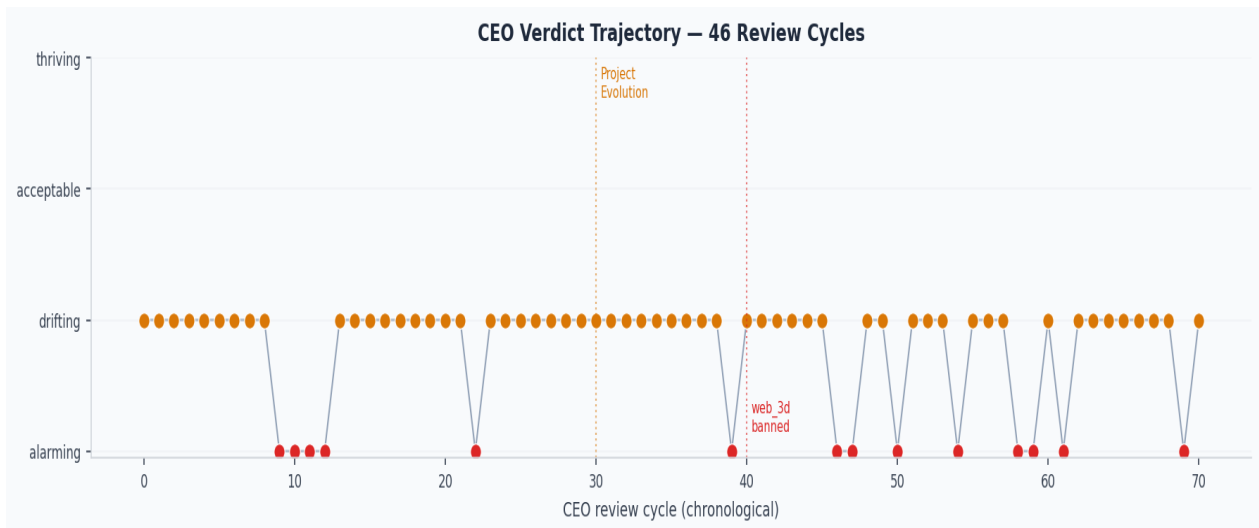


Figure 5. CEO verdict trajectory across 46 review cycles. Key events annotated. The 'alarming' verdict at cycle 40 triggered the successful self-healing pivot.

5. Emergent Behaviours

Several of the most interesting observations were behaviours I did not explicitly program — they arose from the interaction of memory, failure logging, and the multi-agent structure.

5.1 Failure-Driven CEO Strategy Pivots

Early in the project, the CEO had no visibility into refused builds — it could only see what shipped. The result was a CEO that kept demanding ambitious, complex patterns that the QA gate was consistently rejecting, with no ability to course-correct. Adding *failed_builds[]* to the CEO's context changed this immediately. On its next review after seeing a streak of failures, the CEO spontaneously scaled back its complexity demands and shifted domain — without any explicit instruction to do so. This was the first clear sign that the memory architecture was producing genuine emergent learning.

5.2 Complexity Escalation Without Explicit Targets

The complexity floor mechanism sets a minimum, not a target. Architects are free to propose any complexity above the floor. In practice, they consistently propose scores slightly above the floor — typically 1 to 3 points higher. Over 27 projects this produced a compounding escalation that was never directed: no agent was told 'increase by 2 points each time.' The escalation emerged from the combination of the floor rule and the architects' tendency to aim just above the minimum safe threshold.

5.3 Reviewer Disagreement as Quality Signal

Running two reviewers independently at $\text{temperature}=0.85$ means they frequently disagree — one votes *fix*, the other *ship*. The merger logic treats a mixed verdict as *fix*. Across all observed builds, this disagreement pattern correlated with real quality issues: projects where both reviewers voted *ship* on the first round had a significantly higher QA pass rate than those with a split vote. The diversity of opinion turned out to be a useful signal rather than noise.

5.4 Self-Healing via Type Bans

The most striking emergent behaviour was the resolution of the `web_3d` failure loop. Between May 10 and May 11, the system ran 18 consecutive failed builds — all `web_3d`, all blocked by blank canvas or broken controls. The CEO kept demanding `web_3d` because it had never shipped and appeared as a high-priority gap. There was no programmed escape condition for this scenario.

I introduced the type ban mechanism: three consecutive failures of the same type since the last ship triggers an automatic ban, enforced by the validator (hard block) and communicated to the CEO in the diversity report. On the CEO's next review cycle after the ban was activated, its verdict shifted to *alarming* and its directives explicitly stated 'avoid `web_3d` entirely until after successful shipments in other types reset the failure streak.' The very next build — targeting a document type — shipped on the first attempt with a *shippable* QA verdict, ending the two-day blockage.

What I found notable is that the CEO's language changed spontaneously. It identified the problem, articulated a recovery strategy, and pivoted — all from reading the memory log. The self-healing behaviour was not scripted into its response; it emerged from the combination of failure data visibility and the revised prompt's emphasis on shipping over exploration.

6. Limitations and Future Directions

I want to be candid about what this system cannot do, and where my observations are limited.

The most obvious gap is `web_3d`. Eighteen failures without a single ship suggests the mechanical verifier — which checks canvas pixel content — is poorly suited to WebGL contexts, which may render to a non-default framebuffer. A vision-model screenshot reviewer would likely perform better here than pixel-level blank-canvas detection.

The system's 'memory' is shallow by design — a JSON file with a flat list of project records and a growing `concepts_explored` array. There is no semantic search, no embedding-based similarity check, and no way for the system to reason about *why* a previous approach failed in depth. A vector database storing failure post-mortems could meaningfully improve the recovery speed.

The observation period is 21 days, and I observed only a single instance of the self-healing mechanism activating. More time would be needed to determine whether the pattern generalises or was a fortunate coincidence of timing and model behaviour.

Looking forward, I am interested in three directions. First, extending verification to use a vision model for visual quality assessment — not just pixel-level checks. Second, adding cross-type publishing: Python tools to PyPI, documents to arXiv-style preprint servers. Third, and most ambitiously, exploring whether the system can propose and implement improvements to its *own* pipeline code — a genuine self-modification loop.

7. Conclusion

I set out to answer whether a multi-agent LLM system could continuously create diverse, novel software projects at zero cost, without human prompting, and improve over time. Over 21 days, with 27 shipped projects across five domain types, the answer is — within real limitations — yes.

The most important insight from this work is that **failure is information**. Making refused builds visible to the strategic layer — the CEO — was the single highest-leverage change I made to the system. It converted a blind loop into an adaptive one. The CEO did not need to be told 'you are failing'; it read the data and changed its behaviour. The same principle applies to the type ban: the system needed a mechanism to recognise when it was stuck and force itself out, rather than waiting for a human to notice.

The system is fully operational and open-source at github.com/dipeshrayg/autonomous-brain-engine. Every project it has shipped is publicly accessible with a one-click live demo. It continues to run daily, building projects it chose itself.

I think this represents only the smallest demonstration of what structured multi-agent AI systems are capable of. The architecture is not complex — nine LLM roles, a JSON memory file, a bash watchdog. What makes it work is the combination of adversarial critique, persistent failure memory, and automated quality gates. Those three elements, working together, produce a system that is genuinely more capable than any of its parts.

References

- GitHub Actions documentation — docs.github.com/en/actions
- GitHub Models API — github.com/marketplace/models
- OpenAI (2024). GPT-4 Technical Report. [arXiv:2303.08774](https://arxiv.org/abs/2303.08774)
- Playwright browser automation framework — playwright.dev
- GitHub Pages — pages.github.com
- Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
- Park, J. S. et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023.
- AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. [arXiv:2308.08155](https://arxiv.org/abs/2308.08155)

This paper documents original work conducted and authored by Dipesh Ray between April 28 and May 18, 2026. All statistics are drawn directly from the `live_memory_log.json` of the `autonomous-brain-engine` repository.